

Block Round

Die Mathematik des Projekts

Technische und didaktische Dokumentation der mathematischen Grundlagen des Generators für Kreise, Ellipsen, Kugeln und Ellipsoide, *gerendert mit Minecraft-Block-Texturen*. Für jedes Konzept werden die formale Definition, die geometrische Begründung, die direkte Korrespondenz mit dem Quellcode des Projekts und die praktische Übersetzung in In-Game-Baubefehle (`/fill`, `/clone`, `//sphere`) dargestellt.

Themen: analytische Geometrie · diskrete Rasterung · digitale Topologie · Integralrechnung · lineare Algebra · Trigonometrie · Inventar-Arithmetik · Textur-Mapping · oktaedrische Symmetrie.

Inhaltsverzeichnis

1	Überblick: mathematische Natur des Problems	3
2	Koordinatensystem und das diskrete Voxel-Gitter	4
3	Die Fundamentalgleichung: Kreis und Ellipse	4
4	Euklidischer Algorithmus (Distanztest)	5
5	Bresenham-Algorithmus (Ganzzahl-Mittelpunkt)	6
5.1	Kreisfall	6
5.2	Elliptischer Fall (zwei Regionen)	7
6	Schwellwert-Algorithmus (Eck-Abdeckung)	7
7	Render-Modi: Gefüllt, Dünn, Dick	8
7.1	Gefüllt	8
7.2	Dünn (Umriss)	8
7.3	Dick	9
8	Diskrete Flächenberechnung	9
9	Von 2D zu 3D: die Ellipsoid-Gleichung	10
10	Voxelisierung und Volumenberechnung	10
11	Geometrische Schnitte: Ebenen und der 45°-Diagonalschnitt	11
12	Dick-Modus 3D: Dreidimensionalisierung durch Scheiben	12
13	3D-Kamera: Kugelkoordinaten	13
14	Auto-Zoom: einschließende Kugel und FOV	13
15	Schattierung und Texturmultiplikation	14
16	Übergang: von der stetigen Mathematik zum In-Game-Bau	14
17	Arithmetik des Minecraft-Inventars	16
17.1	Einzelner Slot, einzelner Stack, voller Pack	16
17.2	Lagerung: Truhen, Doppeltruhen, Shulker-Boxen	16
17.3	Referenztabelle: kanonische Kugeln	16

18 Kontur-Overlay und Zählung freiliegender Flächen	17
18.1 Was als freiliegende Fläche zählt	17
18.2 Rand-Detektionsformel	17
18.3 Freiliegende Flächenzahl als Survival-Heuristik	17
19 Schichtweiser Bau (horizontale Zerlegung)	18
20 Optimierung durch oktaedrische Symmetrie (O_h)	19
20.1 Konkrete Befehlsfolge	19
20.2 Zeitkosten-Vergleich	20
21 Blocktexturen und visuelle Komposition	20
21.1 Die sechs Flächen und das UV-Mapping	21
21.2 Blockfamilien und Töne	21
21.3 Gradienten durch Block-Mischung	21
22 Fazit	22
Anhang A: Bau-Referenztabellen und durchgearbeitetes Beispiel	24

1. Überblick: mathematische Natur des Problems

Block Round ist ein Generator für abgerundete Formen, *mit Minecraft-Block-Texturen gerendert*: Kreise und Ellipsen in 2D, Kugeln und Ellipsoide in 3D. Das zentrale Problem gehört zur *diskreten Rasterung*: Gegeben eine analytisch definierte Kurve oder Fläche über $\mathbb{R}$, bestimme welche Zellen eines regelmäßigen Gitters (Pixel in 2D, Voxel in 3D) markiert werden müssen. Jede markierte Zelle in Block Round erhält zusätzlich eine Sechs-Flächen-Textur aus dem Minecraft-Sortiment (Grasblock, Bruchstein, Eichenholzbretter, Quarz, Diamant usw.), sodass die resultierende Figur nicht nur eine mathematische Silhouette ist, sondern eine *baubare Struktur*, die der Nutzer im Survival, im Creative mit `/fill` oder über Sponge Schematic v2 (`.schem`) in WorldEdit, Litematica oder MCEdit importieren kann.

Das Problem hat keine eindeutige Lösung. Verschiedene Einschluss-Kriterien erzeugen verschiedene diskrete Silhouetten, jedes Kriterium hat seine eigene mathematische Begründung. Daher implementiert das Projekt drei verschiedene 2D-Algorithmen (Euklidisch, Bresenham, Schwellwert), drei Render-Modi (Gefüllt, Dünn, Dick) und drei 3D-Schattierungsstile. Die Wahl des Algorithmus hängt nicht nur von der visuellen Glätte ab, sondern auch von der Anzahl der Blöcke, die der Nutzer sammeln muss — eine Kugel von $D = 20$ benötigt 4 189 Blöcke unter Euklidisch, aber bis zu 4 322 unter Schwellwert.

Die Ausführung findet vollständig im Browser statt, ohne Server, was eine zusätzliche Anforderung an *Recheneffizienz* stellt (eine Diamant-Kugel von $D = 32$ muss mit 60 fps gerendert werden, mit 17 156 texturierten Voxeln). Die einbezogenen theoretischen Bereiche sind:

- analytische Geometrie von Kegelschnitten und Quadriken;
- inkrementelle Algorithmen mit Ganzzahlarithmetik (Bresenham);
- digitale Topologie (4-, 6- und 26-zusammenhängende Nachbarschaften);
- Integralrechnung und Zählmaß;
- lineare Algebra (Schnittebenen, perspektivische Projektion);
- Trigonometrie und Kugelkoordinaten;
- kombinatorische Inventar-Arithmetik und Gruppen-Symmetrie.

Neuerungen gegenüber dem Schwesterprojekt Pixel Round. Block Round fügt fünf neue mathematische Belange hinzu, die nur entstehen, wenn die gerenderten Zellen zu *Minecraft-Blöcken* werden: (i) das UV-Mapping der sechs Flächen jedes Voxels; (ii) die Arithmetik des Survival-Inventars (Packs von 64 Items, Ein-

zeltruhen mit 27 Slots, Doppeltruhen mit 54 Slots); (iii) das Kontur-Overlay zum Zählen der freiliegenden Flächen während des Baus; (iv) die Zerlegung einer 3D-Form in horizontale Schichten, da der reale Bau Etage für Etage erfolgt; (v) die Ausnutzung der oktaedrischen Symmetriegruppe O_h zur Reduktion der manuellen Blockplatzierungskosten um den Faktor 8 mittels `/clone`.

2. Koordinatensystem und das diskrete Voxel-Gitter

Die Leinwand ist ein Gitter aus $G_x \times G_y$ Zellen in 2D, erweitert auf $G_x \times G_y \times G_z$ Voxel in 3D. Jede Zelle (i, j) belegt das Einheitsquadrat $[i, i + 1] \times [j, j + 1]$; in 3D belegt jeder Voxel den Einheitskubus. Im stetigen Code befindet sich der **Mittelpunkt** der Zelle bei $(i + \frac{1}{2}, j + \frac{1}{2})$. Diese Konvention ist entscheidend: Den Kreis mit der *Ecke* (i, j) oder mit dem *Mittelpunkt* zu vergleichen, ändert welche Zellen zur Form gehören und damit welche Blöcke der Nutzer platzieren muss.

$$\text{Mittelpunkt der Zelle } (i, j) = (i + \frac{1}{2}, j + \frac{1}{2})$$

Für eine Form vom Durchmesser D erweitert der Code das Gitter auf $D + 2$ Zellen pro Achse (Rand von einer Zelle auf jeder Seite). Damit fallen extreme Pixel (N, S, O, W) nie auf den Matrix-Rand. Der **Mittelpunkt** der Form liegt bei $(G_x/2, G_y/2)$, die *Halbachsen* sind $r_x = D/2$ und $r_y = D/2$. Die Einheitszelle der Leinwand entspricht dem Einheitsblock von Minecraft, sodass die Koordinaten des Algorithmus eins zu eins auf die (x, y, z) -Koordinaten im Spiel abgebildet werden.

Minecraft-Mapping. Wenn der Spieler bei $(0, 64, 0)$ spawnet und eine Kugel $D = 16$ ($r = 8$) auf dem Boden möchte, sollte der Mittelpunkt in Welt-Koordinaten bei $(0, 72, 0)$ (8 Blöcke über der Oberfläche) sein, damit der Südpol den Boden berührt. WorldEdit-Befehl: `//sphere stone 8` stehend an $(0, 72, 0)$.

3. Die Fundamentalgleichung: Kreis und Ellipse

Die gesamte Theorie des Projekts beginnt mit der impliziten Gleichung der Ellipse, zentriert im Ursprung mit Halbachsen r_x und r_y :

$$\left(\frac{x}{r_x}\right)^2 + \left(\frac{y}{r_y}\right)^2 = 1$$

Wenn $r_x = r_y = r$, entartet die Ellipse zu einem **Kreis** vom Radius r . Punkte im Inneren erfüllen ≤ 1 ; Punkte im Äußeren, > 1 . Diese Ungleichung entscheidet, ob ein Voxel zur Form gehört und damit ob der Nutzer dort einen Block platziert.

Verschiebung des Mittelpunkts zu (c_x, c_y) und Auswertung im Zellenmittelpunkt:

$$d_x = \frac{i + \frac{1}{2} - c_x}{r_x}, \quad d_y = \frac{j + \frac{1}{2} - c_y}{r_y}, \quad \text{innen} \iff d_x^2 + d_y^2 \leq 1$$

Minecraft-Mapping. Für einen Survival-Turm möchte der Nutzer typischerweise einen *breiten, flachen Kreis* als Fundament und schrittweise kleinere Kreise nach oben. Jedes Stockwerk ist eine Anwendung der impliziten Ellipsengleichung mit denselben Halbachsen und unterschiedlichem z . Block Round berechnet die 2D-Schicht einmal, der Nutzer dupliziert per `/clone` pro Stockwerk.

4. Euklidischer Algorithmus (Distanztest)

Idee. Für jede Zeile j analytisch die Spalten i finden, die in der Ellipse liegen. Auflösen nach x bei gegebenem y :

$$x = c_x \pm r_x \sqrt{1 - \left(\frac{y - c_y}{r_y}\right)^2}$$

Für eine Zeile j mit vertikaler Verschiebung $d_y = j + \frac{1}{2} - c_y$ ist die *horizontale Halbbreite* der Ellipse in dieser Höhe:

$$dxM = r_x \sqrt{1 - \frac{d_y^2}{r_y^2}}$$

Falls $d_y^2/r_y^2 \geq 1$, schneidet die Zeile die Ellipse nicht und wird übersprungen. Sonst werden alle Spalten i mit $c_x - dxM \leq i + \frac{1}{2} \leq c_x + dxM$ gefüllt. Die ganzzahligen Grenzen ergeben sich aus:

$$lo = \lceil c_x - dxM - \frac{1}{2} \rceil, \quad hi = \lfloor c_x + dxM - \frac{1}{2} \rfloor$$

Begründung. Der Term $+\frac{1}{2}$ resultiert aus der Konvention, dass Zelle i den Mittelpunkt bei $i + \frac{1}{2}$ hat. Die Funktionen $\lceil \cdot \rceil$ und $\lfloor \cdot \rfloor$ konvertieren das stetige Intervall in ganzzahlige Indizes und garantieren, dass nur Zellen einbezogen werden, deren **Mittelpunkt** im Inneren der Ellipse liegt. Diese Formulierung liefert die diskrete Approximation, die der stetigen Kurve am nächsten ist, und damit die glatteste Silhouette. Für kleine Durchmesser kann das Ergebnis allerdings weniger pixel-art-artig wirken — ein Euklidischer Kreis $D = 5$ hat nur 13 Blöcke, der Schwellwert-Variante des gleichen Durchmessers 21.

Minecraft-Mapping. Das Euklidische Kriterium kommt dem am nächsten, was `//sphere` in WorldEdit erzeugt. Für eine Bruchstein-Kugel von $D = 16$ ($r = 8$) nur mit Euklidischer Schale ist das Volumen $V \approx 2\,145$ Blöcke im gefüllten Modus, ≈ 804 im dünnen Modus — der Unterschied zwischen dem Sammeln von 34 Packs (2 176) und 13 Packs (832).

5. Bresenham-Algorithmus (Ganzzahl-Mittelpunkt)

Der 1965 für Strecken veröffentlichte Bresenham-Algorithmus wurde später auf Kreise erweitert und von Pitteway und Kappel auf beliebige Ellipsen verallgemeinert. Seine kennzeichnende Eigenschaft ist der Verzicht auf Gleitkomma-Operationen und Wurzelziehen: Die Bestimmung des nächsten Pixels benutzt ausschließlich **ganzzahlige Additionen und Vergleiche**, mittels einer *Entscheidungsfunktion*, die auswertet, auf welcher Seite der analytischen Kurve der Mittelpunkt zwischen den zwei Pixelkandidaten liegt. Für Minecraft-Bau ist es der Algorithmus mit dem am *wiedererkennbarsten* Pixel-Art-Look — dieselbe stufenförmige Silhouette, die die Community seit 2011 von Hand baut.

5.1 Kreisfall

Für einen Kreis mit ganzzahligem Radius R startet man bei $(x=0, y=R)$ mit dem initialen Entscheidungsparameter:

$$d_0 = 1 - R$$

Bei jedem Schritt (im Oktanten von 0 bis 45°):

$$\begin{cases} \text{wenn } d < 0 : & \text{nächster ist } (x+1, y), \quad d += 2x + 3 \\ \text{wenn } d \geq 0 : & \text{nächster ist } (x+1, y-1), \quad d += 2(x - y) + 5 \end{cases}$$

Begründung. Die Funktion d approximiert in jeder Iteration den Wert der impliziten Kreisgleichung ausgewertet am **Mittelpunkt** zwischen den zwei Pixelkandidaten:

$$F\left(x+1, y-\frac{1}{2}\right) = (x+1)^2 + \left(y-\frac{1}{2}\right)^2 - R^2$$

Das Vorzeichen von d gibt an, ob der Mittelpunkt im Inneren des Kreises liegt (nur horizontal weiter) oder außen (auch y dekrementieren). Die acht Oktanten ergeben sich durch die Symmetrien der Diedergruppe D_4 , was im Code den acht `span(...)`-Aufrufen entspricht. Die resultierende Spur zeigt das stufenartige Muster, das charakteristisch für diskrete Approximationen glatter Kurven ist.

5.2 Elliptischer Fall (zwei Regionen)

Für eine Ellipse mit Halbachsen a, b teilt der Algorithmus den ersten Quadranten in **zwei Regionen**, getrennt durch den Punkt, an dem die Tangente 45° bildet:

$$\text{Region I: } 2b^2x < 2a^2y \quad (|dy/dx| < 1), \quad \text{Region II: andernfalls}$$

Die initialen Entscheidungsparameter sind:

$$p_1 = b^2 - a^2b + \frac{a^2}{4},$$

$$p_2 = b^2\left(x + \frac{1}{2}\right)^2 + a^2(y-1)^2 - a^2b^2.$$

Bei jeder Iteration wird p mit vorberechneten Inkrementen aktualisiert (alle ganzzahlig nach Multiplikation mit 4). Das Ergebnis ist die minimale Arithmetik pro Pixel: *ein Vorzeichentest und zwei Additionen*.

Minecraft-Mapping. Für Survival-Bauer ist die Bresenham-Silhouette oft der Euklidischen vorzuziehen, weil die diskreten Schritte leicht manuell zählbar sind — der Nutzer weiß, dass der nächste Block immer entweder *gerade* oder *diagonal* ist. Ein Bresenham-Kreis $D = 11$ hat die kanonische 80-Block-Sequenz aus Dutzenden von Community-Guides.

6. Schwellwert-Algorithmus (Eck-Abdeckung)

Dieser Algorithmus fragt: *Liegt irgendeine Ecke der Zelle in der Ellipse?* Für jede Zelle (i, j) findet der Code die zum Form-Mittelpunkt nächstliegende Ecke:

$$n_x = \text{clamp}(c_x, i, i+1), \quad n_y = \text{clamp}(c_y, j, j+1)$$

$$\text{innen} \iff \left(\frac{n_x - c_x}{r_x} \right)^2 + \left(\frac{n_y - c_y}{r_y} \right)^2 \leq 0,94$$

Der Wert **0,94** ist ein *empirischer Schwellwert* leicht unter 1,0. Sein Zweck ist es, das als *tangentialer Spike* bekannte 1-Pixel-Artefakt an den vier Himmelsrichtungen zu eliminieren, wo die Kurve eine ganze Zellkante tangiert.

Unter den drei Algorithmen liefert dieser die Silhouette mit der größten Fläche bei gleichem D , da die Einschluss-Bedingung am wenigsten restriktiv ist. Für Minecraft liefert er den *rundesten* Look auf Kosten von mehr Blöcken — $D = 16$ liefert 213 Schalen-Blöcke unter Schwellwert gegen 200 unter Bresenham und 192 unter Euklidisch.

Minecraft-Mapping. Schwellwert ist die Wahl, wenn der Bau aus großer Distanz betrachtet wird (z.B. eine Quarz-Kuppel auf einem Tempel), weil die etwas größere Silhouette durch Umgebungs-Verdeckung besser lesbar ist. Trade-off: ca. 5–10% mehr Blöcke pro Schale gegenüber Euklidisch.

7. Render-Modi: Gefüllt, Dünn, Dick

Mit einer Matrix $f[j][i] = 1$ für innere Zellen leitet das Projekt drei visuelle Stile per **diskreter Topologie** ab:

7.1 Gefüllt

Liefert f unverändert. Alle inneren Zellen werden Blöcke. Das produziert `//sphere stone 8`.

7.2 Dünn (Umriss)

Eine Zelle gehört zum dünnen Umriss, wenn sie gefüllt ist *und* mindestens einen orthogonalen Nachbarn (N, S, O, W) *außerhalb* der Form hat. In logischer Notation:

$$b(i, j) = f(i, j) \wedge (\neg f(i-1, j) \vee \neg f(i+1, j) \vee \neg f(i, j-1) \vee \neg f(i, j+1))$$

Geometrisch: das ist der **diskrete Rand** der Form, diskretes Analogon der topologischen Grenze $\partial F = F \cap \overline{F^c}$. In Minecraft entspricht es `//hsphere`: nur die Schale wird materialisiert.

7.3 Dick

Der Dick-Modus fügt den Rand-Pixeln die **Diagonal-Brücken** hinzu: innere Zellen, die gleichzeitig einen horizontalen und einen vertikalen Rand-Nachbarn haben. Das schließt die 1-Pixel-Diagonalen, die der dünne Modus offen lässt, nützlich wenn die Kontur in einer Szene stabil wirken muss (keine 45°-Löcher) — besonders wichtig für Glas- und Eis-Kuppeln.

$$\begin{aligned} t(i, j) &= b(i, j) \vee (f(i, j) \wedge h_H \wedge h_V), \\ h_H &= b(i-1, j) \vee b(i+1, j), \\ h_V &= b(i, j-1) \vee b(i, j+1). \end{aligned}$$

Minecraft-Mapping. Für eine Glas-Kuppel $D = 20$ (hohl) liefert der Dick-Modus eine *wasserdichte* Schale von 608 Blöcken gegen ≈ 510 im dünnen Modus — die ~ 100 zusätzlichen Blöcke sind Diagonal-Nähte, die das Lecken von Umgebungslicht verhindern.

8. Diskrete Flächenberechnung

Die Funktion `area2D` **zählt** einfach die in f markierten Zellen. Das ist das *Zählmaß*, das das Riemann-Integral der Indikatorfunktion der Ellipse approximiert:

$$\underbrace{\pi r_x r_y}_{\text{stetige Fläche}} \quad \longleftrightarrow \quad \underbrace{\sum_{j=0}^{G_y-1} \sum_{i=0}^{G_x-1} f(i, j)}_{\text{diskrete Fläche}}$$

Für großes D gilt diskrete Fläche/stetige Fläche $\rightarrow 1$. Für kleines D ist der Unterschied zwischen Algorithmen sichtbar: Schwellwert tendiert zu Überschätzung; Euklidisch bleibt nahe an der idealen Fläche; Bresenham liegt dazwischen.

Minecraft-Mapping. Der Blockzähler-Chip in der Block-Round-UI zeigt genau diese Zahl: 268 für $D = 8$, 904 für $D = 12$, 2145 für $D = 16$. Der Chip drückt die Zählung als N ($Stacks \times 64 + Rest$) aus — z.B. $2145 = 33 \times 64 + 33$, also „33 Stacks plus 33 Items“.

9. Von 2D zu 3D: die Ellipsoid-Gleichung

In 3D ist die Grundform das **Ellipsoid** zentriert in (c_x, c_y, c_z) mit Halbachsen r_x, r_y, r_z . Seine Gleichung verallgemeinert die Ellipse natürlich:

$$\left(\frac{x}{r_x}\right)^2 + \left(\frac{y}{r_y}\right)^2 + \left(\frac{z}{r_z}\right)^2 = 1$$

Wenn $r_x = r_y = r_z$, bekommen wir die **Kugel** zurück. Auswertung im Zentrum jedes Voxels:

$$a_\xi = \frac{\xi + \frac{1}{2} - c_\xi}{r_\xi} \quad (\xi \in \{x, y, z\}), \quad \text{innen} \iff a_x^2 + a_y^2 + a_z^2 \leq 1$$

Minecraft-Mapping. Ein Ellipsoid mit $W = 20$, $H = 10$, $D = 20$ erzeugt eine *abgeflachte Kugel*, ideal für eine unterirdische Kuppel oder ein Stadiondach. Volumen ≈ 2094 Blöcke (33 Stacks). WorldEdit-Befehl: `//ellipsoid stone 10 5 10.`

10. Voxelisierung und Volumenberechnung

Das stetige Volumen des Ellipsoids ist ein klassisches Ergebnis der Integralrechnung, gewonnen durch Integration über Scheiben:

$$V = \frac{4}{3} \pi r_x r_y r_z$$

Die diskrete Version durchläuft jeden Voxel der Box $D_x \times D_y \times D_z$ und inkrementiert einen Zähler, wenn $a_x^2 + a_y^2 + a_z^2 \leq 1$. Das ist der `voxelVolume`-Algorithmus.

Schale vs. Inneres. Im *filled*-Modus zeigt das Projekt nur Voxel mit mindestens einem 6-Nachbarn außerhalb der erhaltenen Menge (also Voxel, die von der Außenfläche oder von der Schnittfläche sichtbar sind). Im *thin*-Modus nur die **Oberfläche** des vollständigen Ellipsoids (1 Voxel Dicke). Im Survival brauchen nur die sichtbaren Flächen ihren dedizierten texturierten Block — die inneren Voxel können billige Füllung sein (Erde statt Diamant), um Ressourcen zu sparen.

Referenztable. Die Tabelle fasst die kanonischen Kugel-Durchmesser zusammen, die in der Minecraft-Community am häufigsten gefragt werden, übersetzt in Blockzahl (V), Packs zu 64 ($P = \lceil V/64 \rceil$) und Doppeltruhen zu 3456 Slots ($DC = \lceil V/3456 \rceil$). Hilft beim Planen der Ressourcen-Sammlung vor dem Bau.

Durchmesser	Blöcke (V)	Packs ($\times 64$)	Doppeltruhen	Hinweis
D=8	268	5	1	kleine Kuppel
D=12	904	15	1	mittlere Kugel
D=16	2145	34	1	Turmspitze
D=20	4189	66	2	Stadiondach
D=24	7238	114	3	große Kuppel
D=32	17156	269	5	Megabau

Volumen im gefüllten Modus berechnet unter dem Euklidischen Algorithmus für $D \in \{8, 12, 16, 20, 24, 32\}$. Werte im dünnen Modus (Schale) sind ungefähr $V_{\text{Schale}} \approx 4\pi r^2$ Blöcke ein-Voxel-dick, also zwischen 25% und 40% des gefüllten Volumens je nach D .

11. Geometrische Schnitte: Ebenen und der 45°-Diagonalschnitt

Block Round erlaubt das Schneiden des Körpers durch **drei Ebenen**. Jeder Schnitt ist eine *lineare Ungleichung*, die Voxel verwirft:

$$\begin{aligned} \text{X-Achse:} \quad x &\geq cut &\Rightarrow \text{verworfen} \\ \text{Y-Achse:} \quad y &\geq cut &\Rightarrow \text{verworfen} \\ \text{Diagonale:} \quad x + y &\geq cut &\Rightarrow \text{verworfen} \end{aligned}$$

Die X- und Y-Ebenen sind senkrecht zu den Koordinatenachsen mit Normalen $(1, 0, 0)$ und $(0, 1, 0)$. Die *Diagonalebene* hat Normale $(1, 1, 0)/\sqrt{2}$ und schneidet 45° in der XY-Ebene: die Familie $x + y = k$. Der Diagonal-Schnitt-Schieberegler hat Reichweite $D_x + D_y$, während X und Y D_x und D_y haben. Der Schnitt ist **proportional**: der Code speichert *cutPct* und berechnet die Grenze neu als

$$cut = cutPct \cdot \max_{\text{Achse}}$$

sodass 50% auf einer Achse 50% bleibt, wenn der Nutzer die Achse wechselt oder die Größe ändert.

Minecraft-Mapping. Der Y-Achsen-Schnitt bei 50% auf einer Kugel erzeugt die klassische *Kuppel* der halben Volumen. Für $D = 16$: $V_{\text{voll}} = 2145$, $V_{\text{Kuppel}} \approx 1095$ Blöcke (≈ 18 Packs). Der Diagonalschnitt bei 50% erzeugt eine *Halbschüssel*-Sektion, häufig für Brunnen-Interieurs und Amphitheater-Sitzreihen.

12. Dick-Modus 3D: Dreidimensionalisierung durch Scheiben

Für den *thick*-Modus in 3D wendet das Projekt den 2D-Dick-Algorithmus auf **alle Scheiben** entlang der drei Achsen an: XY für jedes z , XZ für jedes y , YZ für jedes x . Dann nimmt es die *Vereinigung* der Ergebnisse. Die geometrische Motivation ist Wasserdichtheit: die minimale Menge von Voxeln, die alle Diagonalen verschließen, ohne die Schalendicke zu verdoppeln. Mit $S_z(z)$ als XY -Scheibe bei Höhe z und T als 2D-Dick-Operator:

$$\text{thick3D} = \bigcup_z T(S_z(z)) \cup \bigcup_y T(S_y(y)) \cup \bigcup_x T(S_x(x))$$

Das Ergebnis wird dann mit der vom Schnitt erhaltenen Menge geschnitten. Die zugrunde liegende Theorie ist **digitale Topologie**: der Dick-Operator garantiert *26-Zusammenhang* der Schale, nützlich in Minecraft, wo diagonale Voxel sich nicht durch Flächen berühren — Licht, Wasser und Kreaturen passieren diagonale Lücken.

Minecraft-Mapping. Für eine Unterwasser-Glaskuppel auf einem Ozeanmonument im Survival ist der Dick-Modus zwingend: die Diagonal-Brücken verhindern, dass Wasser durch die Lücken sickert, die der dünne Modus offen lässt. Kosten: Kuppel $D = 20$ im Dick-Modus benötigt ≈ 380 Glasblöcke gegen ≈ 320 im dünnen Modus — 19% mehr Material für vollständige Abdichtung.

13. 3D-Kamera: Kugelkoordinaten

Die 3D-Kamera umkreist das Objekt mit Abstand r und zwei Winkeln — θ (Azimut, in der horizontalen Ebene) und φ (polar, von der vertikalen Achse). Das ist die **physikalische Konvention** der Kugelkoordinaten, und die Umrechnung in Kartesische (was *three.js* erwartet) ist:

$$\begin{aligned}x &= r \sin(\varphi) \cos(\theta), \\y &= r \cos(\varphi), \\z &= r \sin(\varphi) \sin(\theta).\end{aligned}$$

Anfangsposition: $\theta = \pi/4$ (45° , isometrische Perspektive) und $\varphi = \pi/3$ (60° vom Nordpol, leicht über dem Äquator). Mausziehen inkrementiert θ und φ direkt; Rad/Pinch inkrementiert r . Die Einfachheit dieser Parametrisierung erlaubt freie Rotation ohne Quaternionen oder explizite Matrizen.

14. Auto-Zoom: einschließende Kugel und FOV

Wenn der Nutzer die Figurgröße ändert oder die Kamera zurücksetzt, berechnet das Projekt die **ideale Distanz** neu, sodass das Objekt in jedem Winkel ins Viewport passt. Die Idee ist, das Objekt in eine *Kugel vom Radius R* einzuschließen (halbe Diagonale der AABB, Axis-Aligned Bounding Box):

$$R = \sqrt{(D_x/2)^2 + (D_y/2)^2 + (D_z/2)^2}$$

Eine Kugel hat **rotationsinvariante Projektion**, also passt sie, wenn sie in einer Orientierung passt, in alle. Bei vertikalem FOV der Kamera ($fov_V = 35^\circ$ in Bogenmaß) ist die Distanz für eine Kugel vom Radius R :

$$dist_V = \frac{R}{\tan(fov_V/2)}$$

Das horizontale FOV ergibt sich aus dem Leinwand-Seitenverhältnis durch:

$$fov_H = 2 \arctan(\tan(fov_V/2) \cdot aspect)$$

Die finale Distanz ist das Maximum von $dist_V$ und $dist_H$, multipliziert mit 1,20 (20% visueller Rand). Bei geschnittener Figur verkleinert der Code D_x oder D_y auf die Restgröße. Wenn das Eichenbaum- oder Creeper-Easter-Egg aktiv ist, wird die Bounding Box nach oben erweitert.

15. Schattierung und Texturmultiplikation

Die drei 3D-Stile (*Classic*, *Smooth*, *Blocks*) unterscheiden sich durch **multiplikative Faktoren** auf die drei sichtbaren Flächen jedes Voxels (oben, beleuchtete Seite, dunkle Seite). Die Funktion `shadeHex(hex, factor)` parst das Hex in (R, G, B) und multipliziert jeden Kanal mit Clamping in $[0, 255]$ — das Ergebnis ist ein Tönung über die Rohtextur des Blocks:

$$R' = \text{clamp}(R \cdot f), \quad G' = \text{clamp}(G \cdot f), \quad B' = \text{clamp}(B \cdot f)$$

Das ist eine *lineare Approximation* der idealen Lambertschen Beleuchtungsfunktion,

$$I = \max(0, \mathbf{n} \cdot \mathbf{l}) \cdot \text{Textur},$$

wobei die drei Faktoren dem Kosinus des Winkels zwischen der Flächennormale und einer festen Lichtrichtung entsprechen. In *Smooth* sind die Faktoren ähnlich (ähnliche Flächen); in *Classic* unterscheiden sie sich mehr (starker Kontrast, der Look von Vanilla-Minecraft). *Blocks* führt zusätzlich eine geometrische Einrückung ein — konstante Translation jeder Fläche nach innen, um die Grenzen zwischen Voxeln auch dann sichtbar zu machen, wenn benachbarte Voxel dieselbe Textur teilen. Für emissive Blöcke wird der Lambert-Term durch eine Konstante $I = \text{emissive_factor} \cdot \text{Textur}$ ersetzt.

16. Übergang: von der stetigen Mathematik zum In-Game-Bau

Die fünfzehn vorangegangenen Abschnitte beschreiben die *stetig-zu-diskret*-Pipeline, die Block Round mit seinem Schwesterprojekt Pixel Round teilt: von der impliziten Ellipsengleichung bis zur Kamera, die das Ergebnis einfängt. Die verbleibenden sechs Abschnitte beschreiben, was **spezifisch** für Block Round ist — die zusätzliche Mathematikschicht, die durch die Tatsache eingeführt wird, dass die gerenderten Zellen nicht abstrakte Pixel sind, sondern *Minecraft-Blöcke*, jeder mit sechs texturierten Flächen, einem Gewicht im Inventar und einem Zeitaufwand zum Sammeln und Platzieren.

Das sind keine kosmetischen Erwägungen. Die Entscheidung, mit Blocktexturen

zu rendern, ändert das operative Problem: Statt *ein PNG zu exportieren*, muss der Nutzer *die Silhouette in einen Platzierungsplan übersetzen*. Der mathematische Inhalt der nächsten Abschnitte beschäftigt sich genau mit dieser Übersetzung: Inventar-Arithmetik, Flächenzahl-Optimierung, schichtweise Zerlegung und Ausnutzung der Symmetrie zur Reduktion der Kosten von N Platzierungen auf $N/8 + 7$ Befehlsaufrufe.

17. Arithmetik des Minecraft-Inventars

Ein Minecraft-Spieler trägt Items in einem 36-Slot-*Inventar* in einem 9×4 -Raster. Jeder Slot hält bis zu 64 Items desselben Typs (ein *Stack* oder *Pack*). Der Spieler hat auch Zugriff auf *Lagerung*: Einzeltruhe 27 Slots, Doppeltruhe 54 Slots. Die Arithmetik, die eine Zielblockzahl in einen Sammelplan übersetzt, ist daher ein *Aufrundungs-Divisions-Problem*.

17.1 Einzelner Slot, einzelner Stack, voller Pack

Der *Stack* ist die elementare Einheit der Inventar-Buchführung. Jeder Stack enthält höchstens 64 Items. Die Umrechnung von Blockzahl N in Packs ist

17.2 Lagerung: Truhen, Doppeltruhen, Shulker-Boxen

Um zu berechnen, wie viele *Doppeltruhen* ein Bau erfordert, dividiere durch 54 Slots $\times$ 64 Items/Slot = 3 456 Items pro Doppeltruhe:

$$\text{Packs : } N_{\text{packs}} = \left\lceil \frac{N}{64} \right\rceil \quad \text{Doppeltruhen : } N_{\text{dc}} = \left\lceil \frac{N}{3456} \right\rceil$$

Eine *Einzeltruhe* hält $27 \times 64 = 1\,728$ Items. Eine *Shulker-Box* (tragbarer Behälter) hält $27 \times 64 = 1\,728$ Items, kann aber selbst *in* einem Truhenslot gelagert werden, sodass eine mit 54 Shulker-Boxen gefüllte Doppeltruhe bis zu $54 \times 1\,728 = 93\,312$ Items hält. Für Megabauten ist die relevante Kapazitätseinheit die *shulker-beladene Doppeltruhe*.

17.3 Referenztabelle: kanonische Kugeln

Die Tabelle wandelt die kanonischen gefüllten-Kugel-Volumen in Sammelpläne um. Die letzte Spalte schlägt einen Baukontext für jeden Durchmesser vor:

Durchmesser	V (Blöcke)	Packs	Lagerung	Typischer Bau
D=8	268	5	1 DT	Hüttendach, Brunnenkuppel
D=12	904	15	1 DT	Wachturm-Spitze
D=16	2145	34	1 DT	Dorfglocke, Bibliotheks-Kuppel
D=20	4189	66	2 DT	Observatoriums-Kuppel
D=24	7238	114	3 DT	Tempel-Kuppel
D=32	17156	269	5 DT	Kathedral-Kuppel, World-Boss-Arena

Die *Stacks-und-Rest*-Darstellung $N = q \cdot 64 + r$, die der Info-Chip von Block Round zeigt, spiegelt wider, wie das Survival-Inventar partielle Stacks anzeigt: der Spieler sieht q volle Slots plus einen Slot, der r zeigt. Für $D = 16$ druckt der Chip „2 145 ($33 \times 64 + 33$)“, was dem Spieler sagt, 33 Slots vollständig zu füllen plus einen Slot mit 33 Items — genau 34 Slots.

18. Kontur-Overlay und Zählung freiliegender Flächen

Block Round zeichnet standardmäßig einen *schwarzen Kontur* (Highlight-Overlay) an den Kanten jeder sichtbaren Voxelfläche. Visuell ähnelt das dem „F3+G“-Chunk-Grenzen-Stil des In-Game-Debug-Overlays, auf Voxel-Ebene skaliert. Mathematisch implementiert das Overlay die Visualisierung der **Zählung freiliegender Flächen**: eine Zahl, die Survival-Bauer verwenden, um den Fortschritt Block für Block zu validieren.

18.1 Was als freiliegende Fläche zählt

Eine Fläche des Voxels $\mathbf{v}$, geteilt mit dem Nachbarn $\mathbf{n}$, ist *freiliegend*, wenn und nur wenn $\mathbf{n} \notin V_{\text{solid}}$ (der Nachbar ist leer). Jeder solide Voxel trägt zwischen 0 (vollständig begraben) und 6 (isoliert) Flächen bei. Die Gesamtzahl F_{exp} ist das diskrete Analogon der *Oberfläche*.

18.2 Rand-Detektionsformel

Ein Voxel liegt am Rand, wenn mindestens einer seiner sechs Flächen-Nachbarn außerhalb des Soliden ist:

$$b(i, j, k) = f(i, j, k) \wedge (\neg f(i \pm 1, j, k) \vee \neg f(i, j \pm 1, k) \vee \neg f(i, j, k \pm 1))$$

Es ist derselbe Randoperator wie der 2D-dünne Algorithmus, erweitert auf 3D, und das Overlay zeichnet einfach die Würfelkanten-Linien jedes Randvoxels. Für Glas und Eis ist der Standardwert OFF; für Bruchstein, Stein, Diamant und andere undurchsichtige Blöcke ON.

18.3 Freiliegende Flächenzahl als Survival-Heuristik

Für Survival-Bauer ist die relevante Zählung nicht das Gesamtvolumen V , sondern die *freiliegenden Flächen* F_{exp} , weil das das von außen Sichtbare ist. Die Gesamtsumme ist gleich sechsmal die Randvoxel-Zahl minus zweimal die Anzahl

der Flächen-Adjazenzen unter Randvoxeln:

$$F_{\text{exp}} = \sum_{\mathbf{v} \in V_{\text{solid}}} \sum_{\mathbf{n} \in N_6(\mathbf{v})} [\mathbf{v} + \mathbf{n} \notin V_{\text{solid}}]$$

Für eine Kugel $D = 16$ (gefüllter Modus), $V = 2\,145$, $V_{\text{Rand}} \approx 432$, $F_{\text{exp}} \approx 1\,184$ sichtbare Flächen — der Bau braucht nur etwa 432 *sichtbare* Blöcke; die restlichen 1 713 sind Inneres und können mit dem billigsten verfügbaren Block (Erde, Bruchstein) gefüllt werden. Das Overlay erlaubt es dem Bauer, einen fehlenden Block in der Schale sofort zu erkennen.

19. Schichtweiser Bau (horizontale Zerlegung)

In Minecraft schreitet der Bau *in horizontalen Schichten* voran: der Spieler steht auf einer Schicht, platziert die Blöcke der Schicht darüber, dann steigt einen Block höher. Das 3D-Ellipsoid-Problem versteht man also am besten als *Stapel von 2D-Ellipsen*, eine pro Schicht:

$$\text{Schicht}(k) = \{ (i, j) : a_x(i)^2 + a_y(j)^2 \leq 1 - a_z(k)^2 \}$$

Für jede ganzzahlige Schicht k von $-r_z$ bis $+r_z$ ist die Querschnittsfläche selbst eine Ellipse mit verkleinerten Halbachsen $r_x(k)$ und $r_y(k)$:

$$r_x(k) = r_x \sqrt{1 - \left(\frac{k}{r_z}\right)^2}, \quad r_y(k) = r_y \sqrt{1 - \left(\frac{k}{r_z}\right)^2}$$

Das reduziert das 3D-Problem darauf, *den 2D-Algorithmus der vorigen Abschnitte eine Schicht nach der anderen zu berechnen* und die Ergebnisse zu stapeln. Kognitiv ist das eine massive Vereinfachung: Statt drei mentale Koordinaten zu verfolgen, verfolgt der Bauer zwei Koordinaten pro Schicht plus den Schichtindex.

Minecraft-Mapping. Für eine Kugel $D = 16$ ($r_z = 8$) ist die Äquatorial-Schicht ($k = 0$) der vollständige Bresenham-Kreis $D = 16$ (≈ 200 Blöcke). Die nächste Schicht ($k = 1$) ist ein $D = 15,97$ -Kreis. Bei $k = 4$ ist die Schicht ein $D = 13,86$ -Kreis (≈ 148 Blöcke), bei $k = 7$ ein $D = 7,48$ -Kreis (≈ 43 Blöcke). Die Pole ($k = \pm 8$) sind eine einzelne Block-Kappe.

Der Info-Chip von Block Round zeigt diese Schicht-Zählung im 3D-Modus, wenn der Nutzer den Mittel-Hilfslinien-Button (Taste c) aktiviert. Die horizontalen Scheiben werden als durchscheinendes gelbes Kreuz am Äquator und an jeder $r_z/2$ -Unterteilung gezeichnet.

20. Optimierung durch oktaedrische Symmetrie (O_h)

Eine im Ursprung zentrierte Kugel ist invariant unter der Wirkung der **oktaedrischen Gruppe** O_h der Ordnung 48. Die Voxelisierung der Kugel bewahrt eine Untergruppe der Ordnung 48, die von den drei Koordinaten-Reflexionen und den drei $\pi/2$ -Rotationen um jede Achse erzeugt wird. Für praktische Bauzwecke ist die relevante Untergruppe die Ordnungs-8-*Oktant*-Reflexionsgruppe $\mathbb{Z}_2^3$, erzeugt durch $x \mapsto -x$, $y \mapsto -y$, $z \mapsto -z$. Die Voxelmenge in einem Oktanten bestimmt die ganze Kugel:

$$V_{\text{total}} \approx 8 V_{\text{octant}} \implies \text{Reduktion} = \frac{N - N/8}{N} = 87,5\%$$

Das ist die größte mathematische Optimierung, die einem Minecraft-Bauer zur Verfügung steht. Statt manuell N Blöcke zu platzieren, platziert der Bauer $N/8$ Blöcke im positiven Oktanten und gibt dann sieben `/clone`-Befehle (Vanilla) oder `//copy + //paste`-Befehle (WorldEdit) mit geeigneten Reflexionen.

20.1 Konkrete Befehlsfolge

Für eine Kugel $D = 24$ ($r = 12$, $V = 7\,238$ Blöcke) zentriert in Weltkoordinaten $(0, 72, 0)$, baue zuerst den positiven Oktanten ($+x, +y, +z$ -Region, ≈ 905 Blöcke), dann emittiere:

```

/clone 0 72 0 12 84 12 ~~~ filtered minecraft:stone
/clone 0 72 0 12 84 12 -12 72 0 mode:masked (Refl. -x)
/clone 0 72 0 12 84 12 0 60 0 mode:masked (Refl. -y)
/clone 0 72 0 12 84 12 0 72 -12 mode:masked (Refl. -z)
/clone -12 72 0 -1 84 12 mode:masked (Oktant -x, -y)
/clone 0 60 -12 12 71 -1 mode:masked (Oktant -y, -z)
/clone -12 72 -12 -1 84 -1 mode:masked (Oktant -x, -z)
/clone -12 60 -12 -1 71 -1 mode:masked (Oktant -x, -y, -z)

```

Jedes `/clone` kostet etwa 50 ms Serverzeit, also schließen die sieben Reflexionen in weniger als einer halben Sekunde ab. Die manuelle Platzierung der 905 Oktant-Blöcke dauert etwa 15 min im Survival oder 45 s im Creative mit Brush. Die vollständige Platzierung von 7 238 Blöcken ohne Symmetrie würde ≈ 2 h im Survival oder 6 min im Creative dauern.

20.2 Zeitkosten-Vergleich

Sei T_{block} die Platzierungszeit pro Block (typischerweise 1–2 s im Survival, 0,1–0,5 s im Creative) und T_{clone} die Befehlszeit pro Clone (≈ 50 ms). Die Gesamtzeit mit und ohne Symmetrie ist:

$$T_{\text{octant}} + 7 T_{\text{clone}} \ll T_{\text{full}}$$

Für $N = 7\,238$ im Survival ($T_{\text{block}} = 2$ s, $T_{\text{clone}} = 0,05$ s): voll = 14 476 s ≈ 4 h 1 min; Oktant + Clones = 1 810 s + 0,35 s ≈ 30 min. Ein Faktor-8-Speedup, der genau zur Ordnung der Reflexions-Untergruppe passt. Die volle oktaedrische Gruppe O_h der Ordnung 48 könnte prinzipiell weiter reduzieren, erfordert aber eine 45°-rotierte Baufläche, selten den Setup-Aufwand wert.

21. Blocktexturen und visuelle Komposition

Ein Minecraft-Block ist keine flach gefärbte Zelle, sondern ein *Würfel mit sechs texturierten Flächen*. Block Round rendert jedes Voxel mit seinen sechs kanonischen Flächentexturen, die aus dem Vanilla-Resource-Pack abgetastet werden. Mathematisch ist die Texturierungs-Operation ein *UV-Mapping*: jede Fläche des Einheitswürfels wird durch $[0, 1]^2$ -Koordinaten parametrisiert, und die entsprechende Textur wird an diesen UVs abgetastet.

21.1 Die sechs Flächen und das UV-Mapping

Für jede Voxelfläche konsultiert der Renderer das Block-Modell:

$$\text{Fläche} \in \{\text{oben, Seite, unten}\} \quad \text{UV} : [0, 1]^2 \rightarrow \text{pixels}$$

Die meisten Blöcke (Stein, Bruchstein, Diamantblock, Eichenholzbretter) verwenden die *gleiche Textur auf allen sechs Flächen*. Mehrflächige Blöcke (Grasblock, Kürbis, Heuballen, Quarzsäule, alle Holzstämme, Werkbank, Ofen, Bücherregal, TNT) verwenden *unterschiedliche Texturen* oben, an den Seiten und unten: Grasblock hat grünes Oben, Gras-Erde-Seite und reine Erde unten; ein Eichenstamm hat den Holzmaserring oben und unten und Rinde an den vier Seiten. Entscheidend: die *Mathematik, welche Voxel zu platzieren sind*, hängt nicht von der verwendeten Textur ab — die Silhouette einer Diamantkugel und einer Bruchsteinkugel desselben Durchmessers ist identisch. Die Texturentscheidung ist eine *rein ästhetische* Schicht, angewendet nachdem der geometrische Algorithmus fertig ist.

21.2 Blockfamilien und Töne

Für den Survival-Bauer, der aus den Dutzenden verfügbarer Blöcke wählt, ist die relevante Frage nicht der Texturindex, sondern die *Tonalitätsfamilie*: hell/dunkel, warm/kühl, einheitlich/gemustert. Die Tabelle klassifiziert die häufigsten Blöcke in Block-Round-Bauten:

Block	Familie	Ton	Typische Verwendung
cobblestone	Stein	grau	rustikale Mauern, Verliese
stone	Stein	grau	saubere Mauern, Sockel
oak_planks	Holz	beige	Böden, Innenraum-Abschluss
quartz_block	Quarz	weiß	Tempel, moderne Kuppeln
sandstone	Sand	beige	Wüsten, Pyramiden
deepslate	Stein	dunkel	Unterirdisch, Akzente

21.3 Gradienten durch Block-Mischung

Obwohl ein einzelner Blocktyp einen Ton liefert, können *Gradienten* durch Wechsel zwischen zwei oder drei Blocktypen pro Schicht erzielt werden. Eine klassische Technik ist der *Stein-zu-Quarz-Gradient* auf einer Kuppel: untere Schichten Glatter Stein, äquatoriale Schichten eine verrauschte Mischung aus Glatter Stein und Diorit, obere Schichten Quarz. Jede Schicht bleibt vom selben Block-Round-Algorithmus berechnet; was sich ändert, ist das Per-Voxel-Textur-Lookup. Die *Random-Block-Option* von Block Round implementiert eine solche prozedurale

Mischung standardmäßig (Grasoberteil, Erde darunter, vereinzelte Erze in der Mitte, Tiefenschiefer nahe Grundgestein). Die Mathematik ist dieselbe wie die Schichtzerlegung von Abschnitt 19; nur das „Block-Tag“ pro Voxel ändert sich.

22. Fazit

Dieses Dokument beschrieb systematisch die in Block Round verwendeten mathematischen Grundlagen. In etwa tausend Zeilen JavaScript-Code plus den unterstützenden Textur- und Audio-Pipelines integriert das Projekt Beiträge aus mehreren theoretischen Bereichen:

- **Analytische Geometrie:** implizite Gleichungen der Ellipse und des Ellipsoids als primäres Zugehörigkeitskriterium.
- **Klassische Rasterungsalgorithmen:** Bresenham in Mittelpunkt-Formulierung mit Pitteway/Kappel-Erweiterung für Ellipsen.
- **Digitale Topologie:** diskrete Randoperatoren, Konzepte 4-, 6- und 26-Zusammenhang, Scheibenkomposition und Zählung freiliegender Flächen.
- **Integralrechnung:** Volumen des Ellipsoids als Dreifachintegral und sein diskretes Gegenstück (Zählmaß über Voxel).
- **Lineare Algebra:** Schnittebenen, definiert durch lineare Ungleichungen, und dreidimensionale perspektivische Projektion.
- **Trigonometrie:** Parametrisierung der Kamera in Kugelkoordinaten und automatische Distanzanpassung in Abhängigkeit vom FOV.
- **Kombinatorische Inventar-Arithmetik:** Aufrundungs-Divisions-Buchführung von Packs zu 64 und Doppeltruhen zu 3 456 Items, mit der Stacks-und-Rest-Darstellung des Info-Chips.
- **Gruppensymmetrie:** Ausnutzung der oktaedrischen Gruppe O_h und ihrer Ordnungs-8-Reflexions-Untergruppe $\mathbb{Z}_2^3$ zur Reduktion der Baukosten um den Faktor 8 via `/clone`.

Die zentrale Beobachtung, die die Studie formuliert, ist die folgende: Auf einem diskreten Gitter **existiert keine einzige korrekte Darstellung** einer stetigen Kurve. Jeder in diesem Dokument dargestellte Algorithmus entspricht einer eigenen mathematischen Definition des Zell-Einschluss-Kriteriums, und jede Definition produziert eine Silhouette mit eigenen formalen Eigenschaften — Glätte

beim Euklidischen Algorithmus, Stufenmuster bei Bresenham, größere Abdeckung beim Eckabdeckungskriterium. Die Wahl des Algorithmus ist daher eine technische Entscheidung, die die beabsichtigte visuelle Repräsentation, die gewünschten metrischen Eigenschaften und — speziell für Block Round — die Inventar- und Zeitkosten des physischen In-Game-Baus berücksichtigen muss.

Block Round besetzt eine kleine, aber mathematisch reichhaltige Nische im Minecraft-Tooling-Ökosystem: Es liegt zwischen rein geometrischen Tools (WorldEdit, Litematica) und rein visuellen Tools (Resource Packs, Shader) und bietet eine explizite, ableitbare und reproduzierbare Brücke von der stetigen Gleichung zum diskreten Platzierungsplan. Das Schwesterprojekt Pixel Round bietet dieselben Algorithmen ohne die Minecraft-Schicht, geeignet für traditionelle Pixel-Art- und Voxel-Art-Anwendungen. Zusammen veranschaulichen die beiden Projekte, wie dieselbe stetig-zu-diskret-Pipeline radikal verschiedene künstlerische und konstruktive Arbeitsabläufe unterstützt.

Anhang A: Bau-Referenztabellen und durchgearbeitetes Beispiel

Dieser Anhang fasst die am häufigsten benutzten numerischen und Befehlsreferenzen für Block-Round-Nutzer zusammen. Die Daten konsolidieren die Tabellen pro Abschnitt und die durchgearbeiteten Beispiele aus dem gesamten Dokument, organisiert als einseitige Nachschlagereferenz für Bauplanung, Befehlssyntax und ein durchgängiges Beispiel.

A.1 Umfassende Kugel-Referenz (gefüllt, hohl, Packs)

Die Tabelle erweitert die kanonischen Kugel-Tabellen aus Abschnitt 10 und 17 um die Hohlkugel-Variante (*dünnere* Modus) neben dem gefüllten Volumen. Die Hohlschalen-Werte nehmen eine Schale von einem Voxel an und werden unter dem Euklidischen Algorithmus berechnet; der dicke Modus fügt etwa 10–15% mehr Blöcke hinzu, um diagonale Spalten zu schließen (siehe Abschnitt 12).

D	Gefüllt (V)	Hohl (Schale)	Packs gefüllt	Packs hohl
8	268	170	5	3
10	523	316	9	5
12	904	500	15	8
14	1437	744	23	12
16	2145	1042	34	17
18	3052	1338	48	21
20	4189	1648	66	26
24	7238	2400	114	38
28	11494	3296	180	52
32	17156	4332	269	68

Für einen Survival-Bauer ist die *hohl*-Spalte üblicherweise relevant: nur die sichtbare Schale braucht den dedizierten texturierten Block, während das Innere (falls überhaupt erwünscht) mit dem billigsten Material gefüllt werden kann. Die Spalte Packs-gefüllt zeigt den Gesamtvorrat für eine vollständig gefüllte Kugel; Packs-hohl ist das realistische Minimum für eine hohle Kuppel.

A.2 WorldEdit- / Vanilla-Befehlsreferenz

Die Tabelle fasst die WorldEdit- und Vanilla-Minecraft-Befehle zusammen, die für den Bau von Block-Round-Ausgaben im Spiel am relevantesten sind. WorldEdit-Befehle setzen voraus, dass der Spieler einen Zauberstab (standardmäßig Holzaxt)

hält und eine Region ausgewählt hat; Vanilla-Befehle funktionieren ohne Mod, erfordern aber Koordinatenargumente.

Befehl	Mod / Herkunft	Ergebnis
<code>//sphere stone 8</code>	WorldEdit	gefüllte Kugel $r=8$ (Euklidisch)
<code>//hsphere stone 8</code>	WorldEdit	hohle Kugel $r=8$ (nur Schale)
<code>//cyl stone 8 16</code>	WorldEdit	Zylinder $r=8$, $h=16$
<code>//ellipsoid st. 10 5 10</code>	WorldEdit	Ellipsoid $10 \times 5 \times 10$
<code>/fill ... stone</code>	Vanilla	Box mit einem Blocktyp füllen
<code>/clone ... mode:masked</code>	Vanilla	Region an neuen Ort kopieren

Für Litematica-Nutzer ist der Arbeitsablauf anders: anstatt Befehle auszuführen, wird die Schematic-Datei (Block Round exportiert `.schem`, Sponge-Format v2) in das Litematica-Mod geladen, das dann einen durchscheinenden Geist der Figur anzeigt, den der Spieler Block für Block platziert. Das ist das nächstgelegene Survival-Mode-Äquivalent zur WorldEdit-Creative-Erfahrung.

A.3 Durchgearbeitetes Beispiel: D=20-Kuppel aus Quarz auf einem Tempel

Als vollständig durchgearbeitetes Beispiel angenommen, ein Bauer wünscht eine Quarzkuppel von $D = 20$ auf einem bestehenden Steintempel. Die Tempelspitze ist 11×11 Blöcke, zentriert in $(0, 80, 0)$. Die Kuppel ist die obere Hälfte einer Kugel vom Radius $r = 10$, am $Y = 0$ -Plan geschnitten. Der Plan:

1. **Mit Block Round planen.** App öffnen, 3D-Modus, Kugel, Größe $D = 20$, Schnittachse Y bei 50%. Quarzblock im Picker wählen. Info-Chip meldet $V_{\text{Kuppel}} \approx 2\,126$ Blöcke (≈ 34 Packs, 1 Doppeltruhe mit 1 330 Items im zweiten verbleibend).
2. **Ressourcen sammeln.** $34 \text{ Packs Quarzblock} = \lceil 2\,126/4 \rceil = 532$ Quarzblöcke $\times 4$ Netherquarz je = $2\,128$ Netherquarz. Schmelzen oder handeln nach Bedarf.
3. **Algorithmus wählen.** Für eine von unten gesehene Quarzkuppel gibt der Euklidische Algorithmus die glatteste Silhouette; für eine Kuppel auf einem entlegenen Berg liest der Schwellwert-Algorithmus sauberer. Block Round verwendet standardmäßig Euklidisch.
4. **Kuppel positionieren.** Die ebene Fläche der Kuppel muss mit dem Tempel-Top übereinstimmen. Die Bounding Box ist $20 \times 10 \times 20$ Blöcke; das Zentrum

bei $(0, 80, 0)$ platzieren, sodass die ebene (Schnitt-)Fläche bei $y = 80$ und der Kuppelscheitel bei $y = 89$ liegt.

5. **Mit Symmetrie bauen.** Den $+x, +z$ -Quadranten der Kuppel bauen (≈ 530 Blöcke). `/clone 0 80 0 10 89 10 -10 80 0 mode:masked` ausführen, um an x zu spiegeln; dann `/clone -10 80 0 10 89 10 0 80 -10 mode:masked` für die z -Spiegelung. Gesamtdauer: ~ 10 Minuten im Survival.
6. **Polieren.** See-Laternen an Scheitel und an den vier Himmelsrichtungen des Äquators für emissive Beleuchtung hinzufügen (Block Round rendert diese mit ihrer Emissionskarte beim MC-Lichtlevel 15). Optional das Edge-Overlay (Taste `g`) umschalten, um die Kuppelschale auf Lücken zu prüfen, bevor die letzten Blöcke gesetzt werden.

Die Mathematik jedes Schritts ist genau das, was die vorangegangenen zweiundzwanzig Abschnitte beschreiben: implizite Ellipsoid-Gleichung, Euklidische Voxelisierung, Y-Achsen-Schnitt, oktaedrische Symmetriereduktion via `/clone`, Inventar-Arithmetik zur Übersetzung des Volumens in Packs und Truhen. Der Anhang ist daher keine neue Theorie — er ist eine Erinnerung daran, dass jedes theoretische Konzept dieses Dokuments auf eine einzige, konkrete Entscheidung im Bauplanungsworkflow abgebildet wird.

A.4 Tastaturkürzel-Referenz

Die Block-Round-App stellt einen kleinen Satz Tastaturkürzel bereit, die die Eck-Buttons auf der Leinwand spiegeln. Sie sind unten zur schnellen Referenz aufgelistet; die Kürzel sind global (kein Eingabefeld muss Fokus haben) und funktionieren in 2D- und 3D-Modus, sofern nicht anders angegeben.

Taste	Schaltet um	Anmerkungen
<code>G</code>	Kanten-Overlay	schwarze Umrandung auf jeder Voxelfläche
<code>C</code>	Mittel-Hilfslinien	durchscheinendes gelbes Kreuz an Achsen
<code>D</code>	PNG herunterladen	aktuelle Leinwand als PNG-Bild
<code>I</code>	Info-Chip	Blockanzahl und Pack-Aufschlüsselung
<code>M</code>	2D/3D-Umschaltung	wechselt zwischen flach und voxel
<code>S</code>	Ton an/aus	Umgebungs-Block-Platzierungstöne
<code>T</code>	Tag/Nacht-Thema	verdunkelt Hintergrund, hält Kacheln lesbar
<code>Ctrl+Z</code> / <code>Y</code>	Rückgängig/Wiederholen	durchläuft die Figur-Änderungs-Historie

Die Kürzel `g` (Raster) und `c` (Mittel-Hilfslinien) sind während der Bauvalidierung besonders nützlich: `g` zeigt, ob jedes Voxel der Schale an Ort und Stelle ist (ein

fehlender Block bricht das Overlay-Muster), und `c` bestätigt, dass die Schnitte korrekt auf den Figurachsen zentriert sind.

Für das `Ctrl+Z`-Rückgängig: Block Round durchläuft jede figurändernde Bearbeitung (Schieberegler, Renderingmodus, Algorithmus, Form, Achse, Block, Modus-Umschaltung), schließt aber bewusst rein visuelle Umschaltungen aus (Kamera, Edge-Overlay, Mittelkreuz, Thema, Ton). Diese Trennung hält den Undo-Stack mit Bearbeitungen gefüllt, die der Nutzer wahrscheinlich rückgängig machen will.

A.5 Glossar der Schlüsselbegriffe

Schnellreferenz für die technischen Begriffe, die sich durch dieses Dokument ziehen. Jeder Eintrag rekapituliert kurz das Konzept und verweist auf den Abschnitt, in dem es im Detail entwickelt wird.

- **Voxel** — Einheitskubus in 3D, analog zum Pixel in 2D. Jeder Minecraft-Block ist ein Voxel. Siehe Abschnitt 2.
- **Pack (Stack)** — in Minecraft ein Stapel von 64 identischen Items, der einen Inventarslot belegt. Siehe Abschnitt 17.
- **Doppeltruhe** — zwei benachbarte Truhen, zu einem 54-Slot-Container verschmolzen. Hält bis zu 3 456 Items. Siehe Abschnitt 17.
- **Shulker-Box** — tragbarer 27-Slot-Container, der selbst in einem Truhenslot gelagert werden kann. Siehe Abschnitt 17.
- **Schale** — die äußere, ein-Voxel-dicke Schicht einer festen Figur. Die relevante Oberfläche für Survival-Bauer. Siehe Abschnitte 7 und 18.
- **Oktant** — eine der acht Regionen des 3D-Raums, erhalten durch Schnitt der drei koordinatenbezogenen Halbräume. Verwendet in der Symmetrie-Optimierung. Siehe Abschnitt 20.
- **UV-Mapping** — eine 2D-Parametrisierung einer 3D-Fläche, verwendet um ein Texturbild auf jede Fläche eines Voxels abzubilden. Siehe Abschnitt 21.
- **Bresenham** — inkrementeller Rasterisierungsalgorithmus von 1965, der nur Ganzzahlarithmetik verwendet. Siehe Abschnitt 5.
- **Schwellwert** — Rasterisierungskriterium, das eine Zelle einschließt, wenn eine ihrer Ecken innerhalb der Ellipse liegt. Siehe Abschnitt 6.
- **Euklidisch** — Rasterisierungskriterium, das eine Zelle einschließt, wenn ihr Mittelpunkt innerhalb der Ellipse liegt. Siehe Abschnitt 4.

- **Easter Egg** — eine versteckte Funktion, ausgelöst durch eine spezifische Eingabe. In Block Round: der Eichenbaum und der Creeper, beide bei Schieberreglerwert 15.
- **Schematic** — eine serialisierte Strukturdatei (Sponge Schematic v2, `.schem`), ladbar durch WorldEdit, Litematica oder MCEdit.

Dieses Glossar bleibt absichtlich kürzer als ein vollständiges mathematisches Lexikon. Leser, die eine tiefere Abdeckung der zugrundeliegenden Mathematik suchen (digitale Topologie, Integralrechnung, Gruppentheorie usw.), werden auf die entsprechenden Abschnitte dieses Dokuments verwiesen.

A.6 Ausblick und Grenzen des Modells

Das in diesem Dokument vorgestellte Block-Round-Modell deckt die statische Voxelisierung von Ellipsen, Ellipsoiden und ihren Schnitten sowie die durch die Minecraft-Schicht eingeführten praktischen Überlegungen ab. Es behandelt keine dynamischen Probleme — sich bewegende Figuren, animierte Voxel-Übergänge, Wechselwirkungen mit Fluidsimulation — da diese einen kontinuierlich-zeitlichen Rahmen außerhalb des Geltungsbereichs der diskreten Rasterung erfordern. Eine natürliche Erweiterung ist die *Animation auf Schematic-Ebene*: eine Folge statischer Voxelisierungen, durch Interpolation verbunden, die Block Round bereits als `.schem`-Strom exportieren kann, der von Litematica-Skripten gelesen werden kann.

Eine weitere offene Richtung ist die *exakte volumenerhaltende Voxelisierung*: die aktuelle diskrete Zählung V_{disc} konvergiert nur asymptotisch zum stetigen Volumen V_{cont} . Für Anwendungen, in denen die exakte Zählung der stetigen Formel entsprechen muss (z. B. Rendering-Wettbewerbe, mathematische Kunst mit strengen Blockbudgets), ist eine zusätzliche Verfeinerungsschicht erforderlich — etwa durch leichtes Anpassen des Radius, sodass die diskrete Zählung einen Zielwert trifft. Block Round implementiert diese Verfeinerung derzeit nicht, aber die zugrundeliegende Mathematik ist unkompliziert: $V_{\text{disc}}(r) = V_{\text{Ziel}}$ numerisch nach r lösen.

Schließlich die *Ausnutzung höherer Symmetrie*: das vorliegende Dokument behandelt die Reflexions-Untergruppe der Ordnung 8 von O_h , aber die vollständige Gruppe der Ordnung 48 enthält Rotationen, die im Prinzip eine $\times 48$ -Beschleunigung erlauben würden. Das praktische Hindernis ist, dass In-Game-Befehle in achsenausgerichteten Regionen arbeiten und die Ausnutzung der Rotationssymmetrien eine 45° -gedrehte Baufläche erfordert. Server-Mods, die Auswahlboxen intern drehen (z. B. FAWE), können dies prinzipiell erreichen, aber Mainstream-Tooling stellt es noch nicht zur Verfügung. Eine Referenzimplementierung, die

×48-Beschleunigung auf einem Vanilla-Server demonstriert, wäre ein natürliches Folgeprojekt zur vorliegenden Arbeit.

A.7 Weitere Referenzen und externe Ressourcen

Die in diesem Dokument entwickelten mathematischen Konzepte haben jeweils gut etablierte eigene Literaturen. Für Leser, die Primärquellen oder tiefere algorithmische Hintergründe suchen, werden folgende Einstiegspunkte empfohlen:

- J. E. Bresenham, *Algorithm for computer control of a digital plotter*, IBM Systems Journal, vol. 4 no. 1, 1965 — die ursprüngliche Arbeit zur Ganzzahl-Rasterung, in diesem Dokument erweitert.
- M. L. V. Pitteway, *Algorithm for drawing ellipses or hyperbolae with a digital plotter*, Computer Journal, vol. 10, 1967 — verallgemeinert Bresenham auf beliebige Kegelschnitte.
- A. Kappel, *An ellipse-drawing algorithm for raster displays*, ACM Comm. vol. 28, 1985 — die in Abschnitt 5 verwendete elliptische Zwei-Regionen-Formulierung.
- R. Klette und A. Rosenfeld, *Digital Geometry: Geometric Methods for Digital Picture Analysis*, Morgan Kaufmann, 2004 — Standardreferenz für digitale Topologie und die Randoperatoren der Abschnitte 7 und 18.
- WorldEdit-Dokumentation, enginehub.org/worldedit/ — offizielle Befehlsreferenz für die in diesem Dokument verwendeten Befehle `//sphere`, `//hsphere`, `//ellipsoid`, `//copy`.
- Sponge Schematic Specification v2, github.com/SpongePowered/Schematic-Specification — das Dateiformat, das Block Round als `.schem` exportiert.

Die in Abschnitt 22 aufgelisteten acht theoretischen Gebiete haben jeweils Lehrbuch-Niveau-Behandlungen, die weit über das hinausgehen, was ein einzelnes technisches Dokument zusammenfassen kann. Der Beitrag von Block Round liegt nicht in der Theorie selbst, sondern in der spezifischen Synthese: Anwendung der klassischen Rasterung auf die texturierte Voxel-Minecraft-Umgebung mit den expliziten Inventar- und Symmetriekostenüberlegungen, die in rein geometrischen Tools üblicherweise weggelassen werden.

Eine abschließende Bemerkung zur Reproduzierbarkeit: die gesamte Pipeline dieses Dokuments — Build-Skript, Übersetzungen, LaTeX-Vorlage, Pro-Locale-Tabellen — ist im `docs_math/`-Verzeichnis des Projekt-Repositorys enthalten. Erneutes Ausführen von `python build.py` auf einem System mit XeLaTeX/MiKTeX erzeugt dieses PDF exakt, in jeder der neun unterstützten Sprachen. Das Schwesterprojekt

Pixel Round folgt der gleichen Struktur und ermöglicht den direkten Side-by-Side-Vergleich der texturierten und nicht-texturierten Varianten.

Ende des Dokuments

Block Round wird von Vinícius Rodrigues de Souza gepflegt. Das Schwesterprojekt Pixel Round ist unter github.com/ViniSouza128/pixel-round verfügbar; das aktuelle Block-Round-Repository ist unter github.com/ViniSouza128/block-round.

Kommentare, Korrekturen und Übersetzungsverbesserungen sind über den Issue-Tracker des Projekt-Repositorys willkommen. Korrekturlesen durch Muttersprachler der acht nicht-englischen Locales wird besonders geschätzt, da die Lokalisierungen mit erheblicher Überprüfung vorbereitet wurden, aber die technisch-mathematische Terminologie von muttersprachlicher Korrektur profitiert.